

Adaptive ChiaLoop for Compute-Efficient Coverage Closure in Chip Verification

Kapil Bansal Khushal Sethi

CHIA Hackathon at A3@MICRO 2026

Artifact: <https://github.com/kbansal1512/chia-hackathon>

Abstract—Chip verification must close coverage within compute, license, and schedule budgets, yet fixed schedules can continue spending resources on low-yield work. We present ChiaLoop, an adaptive chip-verification system that replans job allocation using observed coverage gain per compute-hour. Its policy aims to exploit the nonlinear effects of scenario selection by prioritizing high-yield obligations and avoiding jobs proven unable to contribute under registered assumptions. In an exploratory post-saturation comparison across five MediumBOOM trial pairs, Adaptive recorded 169.2 mean new-bin hits per trial versus 160.6 for a fixed 4:1 constrained-random-to-directed schedule. This preliminary result motivates broader evaluation. ChiaLoop is designed to coordinate allocation and budgets across users, chip configurations, license constraints, and AI agents.

Index Terms—adaptive verification, coverage closure, resource allocation, constrained-random verification, formal verification, CHIA

I. INTRODUCTION

Verification allocates limited execution capacity across complementary methods. Constrained-random tests sample diverse behaviors, directed tests concentrate on specific scenarios, and formal queries produce witnesses or establish unreachability under explicit assumptions. Their usefulness changes with the remaining obligations. An allocation effective early in a campaign can become wasteful near its closure target.

Coverage-directed test selection has demonstrated the value of using feedback to prioritize simulation work [1]. A broader campaign also chooses among verification engines and coverage partitions. The resulting question is operational: given the current evidence and available resources, which eligible job should receive the next compute slot? Increasing parallelism alone does not resolve this choice. It may shorten elapsed time while executing the same amount of redundant work.

We propose *ChiaLoop*, a verification controller hosted on the CHIA execution framework [2]. ChiaLoop treats the campaign objective as an explicit input. In the core study, the objective is to reduce occupied compute-hours required to reach a registered sound-closure target. The controller measures the marginal closure obtained from completed jobs, updates recent yield estimates, and selects further work subject to compute and license constraints. Its control decisions depend on observed outcomes rather than fixed coverage-percentage thresholds for switching engines.

The research question is: *Does completion-driven allocation across constrained-random, directed, and formal work reach 95% sound coverage closure using fewer compute-hours than*

TABLE I
COVERAGE EVIDENCE AND ACCEPTANCE RULES

Evidence	State	Required interpretation
Simulation observation	Hit	Native evidence for the registered predicate and scope.
Formal cover witness	Hit	Legal witness under matching model and assumptions.
Complete	Proven	Exact obligation, qualified scope, and registered assumptions.
unreachability proof	unreachable	No evidence sufficient to close the point.
Bounded non-hit; timeout; unknown; error	Open	
Conflicting or incompatible result	Unchanged	Reject and log; investigate the campaign's validity.

a fixed hybrid schedule, when jobs, resources, and evidence rules are held constant? A fixed hybrid is the primary control because it already has access to every engine used by the adaptive treatment. This comparison separates allocation quality from the benefit of adding formal or directed verification to a simulation campaign.

This paper makes three methodological contributions. First, it defines an objective and evidence model that couples compute accounting with a fixed universe of coverage obligations. Second, it specifies a deterministic adaptive policy and asynchronous execution architecture with explicit resource admission. Third, it reports an exploratory post-saturation comparison using five MediumBOOM trial pairs, with the earlier single-pair pilot and hindsight replay kept separate.

II. PROBLEM FORMULATION AND EVIDENCE MODEL

Let P be the frozen coverage universe and J the immutable job catalog. A point is open, hit, or proven unreachable; sound closure is $(|H| + |U|)/|P|$ with a fixed denominator. Simulation hits and formal outcomes must match the registered predicate and scope. Table I summarizes the evidence rules.

The driver derives novelty at merge time. Duplicate hits earn no credit; conflicting evidence is quarantined (Fig. 1).

The objective is occupied slot-hours to 95% sound closure: $Q(t) = \int_0^t \sum_s a_s(u) du$ and $\text{Compute}_{95} = Q(T_{95})$. All occupied work, including failures and timeouts, counts toward cost.

III. CHIALOOP ARCHITECTURE AND ADAPTIVE POLICY

A. Separation of Policy, Execution, and Evidence

Figure 2 shows the proposed control path. The campaign specification fixes the objective, coverage universe, job cata-

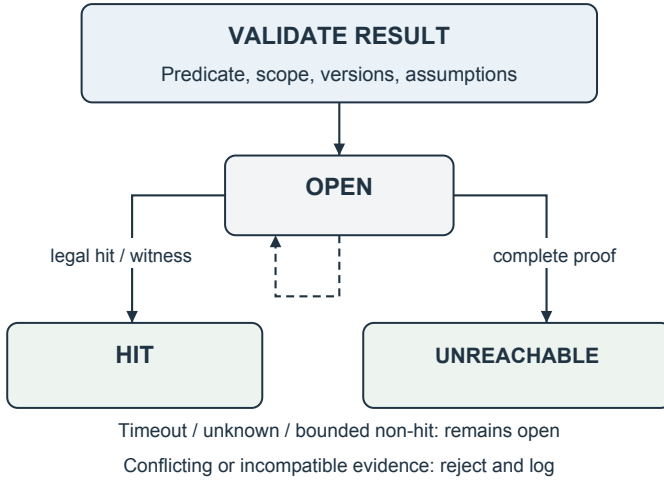


Fig. 1. Evidence-governed state updates. Validation precedes every transition. Incomplete evidence leaves an obligation open; a conflicting result is quarantined without awarding additional closure.

log, capacities, and stopping rules. The controller owns the coverage state and yield table. A replaceable policy chooses jobs, while the dispatcher admits and launches them through three CHIA function nodes: `run_crv`, `run_directed`, and `run_formal`. Each invocation executes one independently schedulable job.

The registered toggle campaigns contain CRV and directed simulation jobs only, with two compute slots, one simulation token, and no formal jobs. The endpoint is exact signal-toggle bits; SVA/property outcomes cannot soundly hit or retire those toggle IDs, so formal jobs are excluded rather than mixed into this comparison. A declared formal-license capacity does not imply formal work was executed. These campaign-controlled tokens express concurrency constraints; an open-source tool run does not imply possession of a commercial license.

After probing each eligible lane once, Adaptive ranks lanes by new validated points per occupied compute-hour over the last three completed jobs. Every tenth post-probe dispatch explores the least-sampled eligible lane; ties and job order are frozen. The asynchronous driver validates completions, merges unique hits, updates yield, admits work only when resource tokens fit, and records decisions and costs in an append-only event log. Static-Hybrid uses the same execution and evidence path with a fixed lane schedule.

IV. EXPERIMENTAL METHODOLOGY

A. Workload and Frozen Inputs

We use one MediumBOOM configuration [3] and five fixed planning partitions: integer/control, load-store/ordering, multiply/divide/atomics, privilege/exceptions, and mixed long-running behavior. Inputs and coverage IDs are frozen before paired runs; duplicate point hits count once. Table II summarizes the five-pair cohort.

The registered endpoint is native signal-toggle evidence; the measured toggle catalog contains CRV and directed jobs but

TABLE II
CAMPAIGN CONFIGURATION AND EXPERIMENTAL CONTROLS

Item	Core-study specification
Design and planning groups Engines	One MediumBOOM configuration; five partitions. CRV and directed simulation in the registered toggle study; formal is excluded from this endpoint.
Execution capacity Admission tokens	Two identical compute slots. One simulation token; no formal jobs are catalogued.
Closure target Adaptive parameters	95% of the frozen point universe. Three completed jobs per window; exploration every tenth post-probe dispatch.
Static manifest Pairing	Repeating 4 CRV : 1 directed schedule. Identical jobs and initial state within each pair; randomized arm order.
Execution registration	The frozen cohort protocol and manifests are archived with the measured results.

no formal jobs. Source-line, semantic-family, and proof results are not mixed into this endpoint.

B. Primary Baseline and Treatment

Static-Hybrid follows a fixed 4 CRV:1 directed schedule; Adaptive changes only dispatch order. Each pair uses the same frozen jobs, evidence validator, initial state, resources, and stopping rule, with randomized arm order and no result replay. The comparison runs through a local asynchronous executor, not CHIA/Ray; historic proxy-meter counts are excluded.

V. RESULTS AND ANALYSIS

A. Exploratory Capped-Concurrency Open-Toggle Comparison

The endpoint is native Verilator signal-toggle bins left open after the prior pilot; neither the legacy 51-point proxy nor the 66 semantic families enters the denominator. This is toggle activity, not source-line or functional coverage.

Summed per-trial new-bin hits were 803 for Static-Hybrid and 846 for Adaptive, or 160.6 and 169.2 per trial; the paired summed difference was +43 hits (+8.6 per trial). The post-saturation log curves project additional residual-set closure of 11.09% and 11.55% at 200 catalogs, respectively, a +4.2% relative difference in projected incremental gain. These values extrapolate from only five measured catalogs; they are exploratory projections, not observed outcomes or evidence of a confirmatory policy advantage.

Workloads are pre-generated; Adaptive changes dispatch order only and does not synthesize directed tests on the fly.

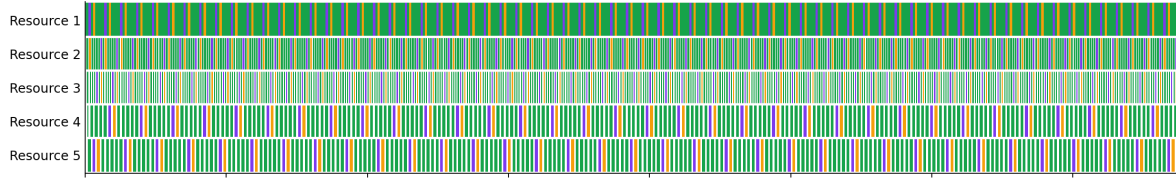
The five-pair subset is exploratory; its 200-catalog log values are projections, not observed Compute95. The toggle catalog has no formal jobs, so these data neither establish proof-based closure nor a reliable general policy advantage. Initialization cost, job duration, and feedback delay remain potential explanations for observed differences.

Resource allocation · Static vs Adaptive

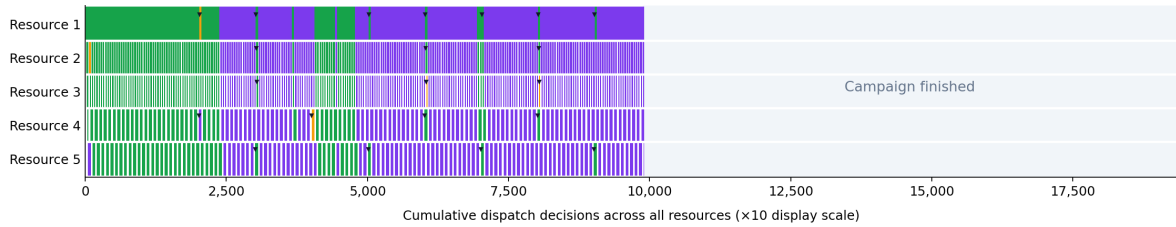
Latest smoke pair · 5 resource slots per arm · 98% coverage target

CRV 1-5 Directed Formal Neglected-node retry

Static · 1,933 actual dispatches



Adaptive · 989 actual dispatches · 21 retry dispatches



Colors identify engines; all five CRV nodes share green. Black triangles mark the 100-decision retry rule. Each column is resource allocation at a dispatch decision, including jobs awaiting result processing. Width does not represent runtime.

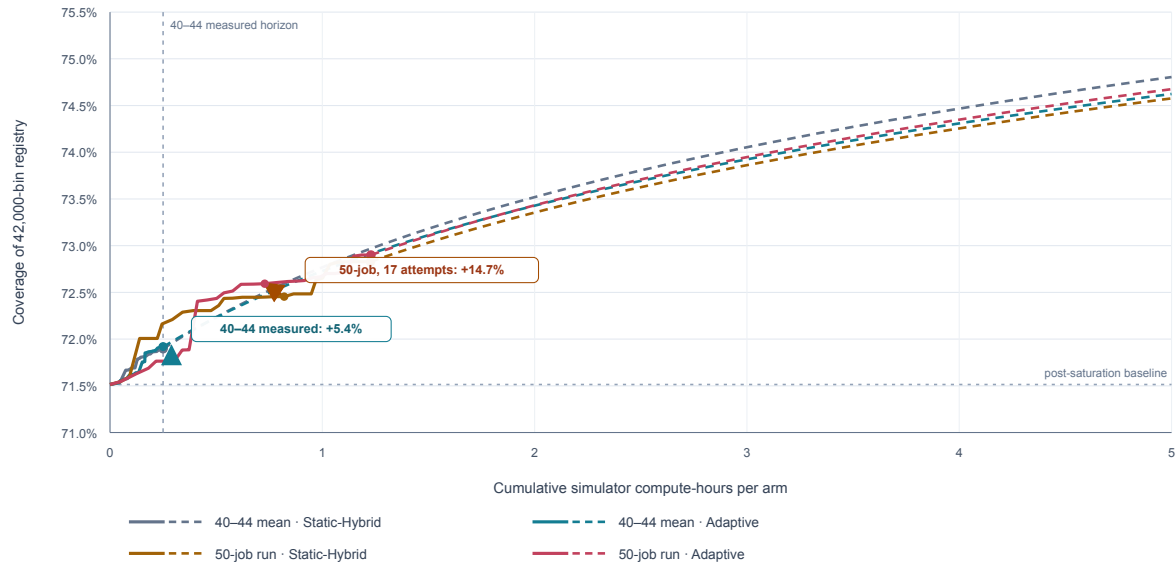
(a) Illustrative static and adaptive resource allocation across five slots. This separate trace includes formal work and a 98% target.

MediumBOOM toggle coverage vs compute time

Trials 40–44 (five-run mean) vs one profiled 50-job CRV:Directed 4:1 run

Measured checkpoint deltas: 40–44 +5.4%; 50-job at 17 attempts +14.7% (shared toggles)

Interim 50-job snapshot: static 14 successes/3 timeouts; adaptive 17/0. Dashed 5h fits are exploratory.



At equal 17 attempts, the full 12,156-point endpoint was 568 static vs 621 adaptive (+9.3%); static timeouts count as attempts. Measured $\Delta\%$ = $100 \times (\text{Adaptive} - \text{Static}) / \text{Static}$ gained bins. Shared 11,964 native-toggle scope shown; dashed 5h log fits are exploratory only.

Fig. 3. Resource allocation and coverage. (a) The separate five-slot trace is illustrative, not part of the reported 95% CRV/directed evaluation. (b) Post-saturation coverage versus compute-hours: trials 40–44 are the five-trial mean to 0.25 hours (+5.4% Adaptive); the 50-job run shows the interim equal-17-attempt checkpoint (+14.7%, 453 versus 395 bins). Five-hour log continuations are exploratory projections, not observed outcomes, and may cross.